

PROGRAMACIÓN I

ALGORITMOS Y ESTRUCTURAS DE DATOS I

Ruptura de ciclos y excepciones

`break` · `continue` · `else` en ciclos · `while True` · `try/except`

Ing. Maria Eugenia Varando

La ruta de la clase

Retomamos estructuras iterativas y aprendemos a gestionar errores sin “romper” el programa

01 Ruptura de ciclos

break, continue y cuándo conviene usarlos

03 while True y pass

ciclo infinito controlado, condición de corte y marcador vacío

02 else en ciclos

for-else y while-else como cierre normal de iteraciones

04 Excepciones

try/except, else, finally, raise, assert y excepciones propias

Regla de oro para esta clase

No miramos solo qué imprime: miramos por dónde circula el programa



Predigo

¿El ciclo termina, continúa o saltea esta vuelta?



Ejecuto

Corro el caso de prueba y observo el recorrido.



Comparo

Busco diferencias entre lo esperado y lo real.



Explico

Pongo en palabras la condición que activó el corte o la excepción.



Refactorizo

Reordeno el código para que la intención quede clara.

Pregunta recurrente: “¿qué línea se ejecuta ahora y por qué?”



Bloque 1

Ruptura de ciclos

break y continue cambian el flujo normal de una iteración. Los usamos con intención, no como atajos para ocultar errores.

break y continue

Instrucciones que rompen o alteran la ejecución normal de ciclos while / for

break

Finaliza por completo el ciclo.
El programa continúa en la primera línea posterior al while o for.

continue

Finaliza la iteración actual.
El ciclo sigue con la siguiente vuelta.

Ambas instrucciones pueden utilizarse las veces que sea necesario dentro de un ciclo, pero conviene justificar por qué están ahí.

continue

Cuando se cumple la condición, salta a la siguiente iteración

Patrón de lectura

```
for elemento in secuencia:  
    if condicion_para_salto:  
        continue  
    # solo se ejecuta si NO saltó  
    procesar(elemento)
```

Idea central

continue no termina el ciclo: solo corta la vuelta actual y vuelve a evaluar la condición del ciclo o avanza al próximo elemento.

Ejercicio base: desarrollar una función para sumar los dígitos pares.

Ejemplo continue: sumar dígitos pares

Versión simple recorriendo los caracteres de `str(abs(nro))`

```
def suma_digitos_par(nro):  
    """Suma los dígitos pares."""  
    suma = 0  
    for digito in str(abs(nro)):  
        if int(digito) % 2 != 0:  
            continue  
        suma += int(digito)  
    return suma
```

Lectura paso a paso

- Convierte el número a texto para recorrer dígito por dígito.
- Si el dígito es impar, continue saltea la suma.
- Si el dígito es par, se acumula.

Pregunta: ¿qué devuelve con 4527?

Ejemplo continue: sin convertir todo el número a texto

Otra forma de resolver el mismo problema con división entera y módulo

```
def suma_digitos_par2(nro):  
    """Suma los dígitos pares."""  
    suma = 0  
    if nro < 0:  
        nro = nro * (-1)  
    while nro > 0:  
        digito = nro % 10  
        nro = nro // 10  
        if digito % 2 != 0:  
            continue  
        suma = suma + digito  
    return suma
```

¿Dónde actúa continue?

Si el dígito es impar, se saltea la línea
suma = suma + digito y vuelve a
evaluar el while.

El ciclo sigue; lo que cambia es la vuelta actual.

break

Cuando se cumple la condición, finaliza por completo el ciclo

Patrón de lectura

```
while condicion_general:  
    if condicion_de_corte:  
        break  
    seguir_procesando()  
  
# el programa continúa acá
```

Idea central

break termina el ciclo completo, no solamente una iteración. Es útil cuando ya encontramos lo que buscábamos o apareció una condición de salida.

Ejercicio base: desarrollar una función para determinar si un número es primo.

Ejemplo break: determinar si un número es primo

El ciclo se corta apenas encuentra un divisor

```
def es_primo(numero):  
    """Retorna True si el número es  
    primo,  
    False en caso contrario."""  
    if numero <= 0:  
        return False  
  
    primo = True  
    for divi in range(2, numero):  
        if numero % divi == 0:  
            primo = False  
            break  
    return primo
```

Lectura

- El for prueba divisores posibles.
- Si aparece uno que divide exacto, ya no hace falta seguir.
- break corta el recorrido.

**Pregunta: ¿cuántas vueltas hace con
numero = 9?**

for-else

La estructura for puede tener un else asociado

Cuándo se ejecuta

El else se ejecuta si el for finaliza de manera “normal”. Normal significa: terminó de recorrer todos los iterables.

```
for elem in datos:  
    if condicion_de_corte:  
        break  
else:  
    print("No hubo corte")
```

si aparece break, el else no se ejecuta.

while-else

La estructura while también puede tener un else asociado

Cuándo se ejecuta

El else se ejecuta si el while finaliza de manera “normal”. Normal significa: su condición resultó falsa.

```
while condicion:  
    if condicion_de_corte:  
        break  
else:  
    print("La condición del while terminó en False")
```

El else del ciclo no significa “si no entró al while”: significa “si no se cortó con break”.

Ejemplo while-else: búsqueda en lista

El else informa que el valor no fue encontrado si el ciclo terminó normalmente

```
lista = [3, 23, 45, 18, 32, 8]
nro = int(input("Ingrese un número a buscar: "))
i = 0
while i < len(lista):
    if nro == lista[i]:
        print("El nro se encontró en la ubicación", i)
        break
    i = i + 1
else:
    print("El nro no se encontró")
```

Interpretación

- Si encuentra el número, ejecuta break.
- Si recorre toda la lista sin encontrarlo, la condición del while se vuelve falsa.
- Ahí se ejecuta el else.

while True: ciclo infinito controlado

Se usa cuando la condición de corte está dentro del bloque iterativo

Definición

while True es un bucle infinito a menos que exista una condición para terminarlo dentro del bloque. Si no existiera condición de terminación, se ejecuta en forma indeterminada.

```
while True:  
    # ...  
    if condicion == False:  
        break
```

**cargas con banderas o condicion de fin, menues o validaciones
que deben repetirse hasta una entrada válida.**

Ejemplo while True: carga con condicion de fin

El ciclo se repite hasta ingresar -1

```
suma = 0
while True:
    nro = int(input("Ingrese un
número: "))
    if nro == -1:
        break
    suma = suma + nro
print("La suma es", suma)
```

Cómo se lee

- La condición externa es siempre verdadera.
- El corte real está dentro del ciclo.
- -1 no se suma: corta antes de acumular.

while True no reemplaza al for

Usarlo para emular un ciclo for es una mala práctica de programación

Mala práctica

```
# Evitar: simula un for
i = 0
while True:
    if i >= 10:
        break
    print(i)
    i += 1
```

Más claro

```
# Mejor: la intención es clara
for i in range(10):
    print(i)
```

Criterio: si sé cuántas veces o qué secuencia voy a recorrer, for suele expresar mejor la intención.

Instrucción pass

Un marcador vacío para mantener la estructura sintáctica

```
def validar_datos():  
    pass  
  
for elemento in lista:  
    if elemento < 0:  
        pass # pendiente de definir  
    else:  
        print(elemento)
```

Idea de uso

pass no hace nada. Se usa cuando necesitamos que exista un bloque, pero todavía no vamos a escribir su lógica. Sirve como “pendiente” sin romper la sintaxis.

No confundir con continue: pass no saltea la iteración, solo no ejecuta ninguna acción.



Bloque 2

Excepciones

Una excepción es un problema ocurrido durante la ejecución de un programa. Aprendemos a capturarlo, explicarlo y decidir qué hacer.

¿Qué es una excepción?

Un problema que aparece mientras el programa se está ejecutando

Definición

Una excepción es un problema ocurrido durante la ejecución de un programa.

```
suma = 15  
cant = 0  
promedio = suma / cant  
  
# ZeroDivisionError: division by zero
```

El error no está en escribir mal la sintaxis: el problema aparece al ejecutar con ciertos datos.

Excepciones conocidas

Errores frecuentes que vamos a reconocer leyendo el mensaje

Durante ejecución

- IndexError
- NameError
- ValueError
- ZeroDivisionError

Sintaxis / uso del lenguaje

- SyntaxError
- IndentationError
- TypeError
- AttributeError

Aprender a leer el tipo de excepción ayuda a ubicar rápidamente dónde mirar.

Otras excepciones

También pueden aparecer errores asociados a claves, memoria, módulos o interrupciones

AssertionError
IOError
KeyError
KeyboardInterrupt
MemoryError
ModuleNotFoundError
RecursionError

No necesitamos memorizarlas todas: sí necesitamos leer qué excepción aparece y en qué línea.

Dos estrategias frente a los errores

Control preventivo y captura correctiva

Control de errores tradicional

- Parte del código se dedica a controlar posibles situaciones de error.
- Actúa de forma preventiva: evita que el error ocurra.

Captura de excepciones

- La lógica del programa es independiente del control de errores.
- Actúa de forma correctiva: intenta solucionar el problema luego de ocurrido.

No son enemigos: se elige según claridad, legibilidad y tipo de problema.

Control tradicional vs captura de excepciones

Mismo problema, dos formas de resolverlo

Control tradicional

```
# Preventivo
if cant != 0:
    print(f"El promedio es:
{suma/cant:4.2f}")
else:
    print("La cantidad ingresada es 0")
```

Captura de excepciones

```
# Correctivo
try:
    print(f"El promedio es: {suma/cant:4.2f}")
except ZeroDivisionError:
    print("La cantidad ingresada es 0")
```

La captura permite separar la intención principal del tratamiento del error.

Estructura try-except

El bloque protegido contiene código que puede generar errores

```
try:  
    <bloque de código que puede generar errores>  
    ...  
except <Excepción a capturar>:  
    <bloque de código para tratar el error generado>  
    ...
```

Bloque protegido

Es el código que queremos intentar ejecutar.

Si allí ocurre un error esperado, Python salta al except correspondiente.

Funcionamiento de try-except

Qué pasa cuando hay error y qué pasa cuando no hay error

1 Se ejecuta el try

Primero se ejecuta el bloque protegido.

3 Si no hay error

El bloque except se ignora.

2 Si hay error

Se pasa automáticamente al except que corresponde a la excepción generada.

4 Múltiples except

Puede haber tantos except como sea necesario, sin repetir tipo de excepción.

Funcionamiento: except sin tipo

Captura cualquier excepción, pero no es recomendable

Evitar cuando se pueda

```
try:  
    dato = int(input("Ingrese un número: "))  
except:  
    print("Ocurrió un error")
```

Por qué no conviene

El except puede no estar seguido de un tipo de excepción. En ese caso captura cualquier excepción generada, aunque no es recomendado porque no permite identificar el error.

Ejemplo con ValueError e IndexError

Una entrada inválida y una posición fuera de rango requieren tratamientos distintos

```
import random
lista = [random.randint(1,20) for i in range(random.randint(5,10))]
try:
    posicion = int(input("Ingrese posición del elemento a mostrar: "))
    print(f"El valor en la posición {posicion} es: {lista[posicion]}")
except ValueError:
    print("No se ingresó un número")
except IndexError:
    print("La lista no tiene tantos elementos")
```

Lectura

- ValueError: el int() no pudo convertir.
- IndexError: la posición no existe en la lista.

Otras opciones del except

Un mismo tratamiento para más de una excepción

```
try:
    cant = int(input("Ingrese la cantidad: "))
    promedio = suma / cant
except (ValueError, ZeroDivisionError):
    print("El valor ingresado no es correcto")
```

Regla

Si a más de una excepción se le va a dar el mismo tratamiento, puede colocarse más de un tipo de excepción en la misma cláusula except.

Capturar el mensaje de error

La instrucción `as` permite guardar el mensaje asociado a la excepción generada

```
try:  
    num = int("aaa")  
except ValueError as msg:  
    print(msg)  
  
# invalid literal for int() with base 10: 'aaa'
```

¿Para qué sirve?

Permite registrar, mostrar o reenviar información más específica sobre el error capturado.

Cláusula else en try-except

Se ejecuta solo si el bloque protegido no generó errores

```
try:
    porcentaje = (cant / total) * 100
except ZeroDivisionError:
    print("No puede calcularse el porcentaje")
else:
    print(f"El porcentaje es {porcentaje:5.2f}%")
```

Lectura

Luego de las cláusulas except, puede agregarse else. Su bloque se ejecuta únicamente si el try terminó sin generar ningún error.

Cláusula finally

Se ejecuta siempre, haya ocurrido o no un problema

```
try:
    porcentaje = (cant / total) * 100
except ZeroDivisionError:
    porcentaje = 0.0
finally:
    print(f"El porcentaje es {porcentaje:5.2f}%")
```

Uso típico

finally se utiliza para acciones que deben ejecutarse siempre: cerrar recursos, mostrar un cierre o limpiar estados.

Instrucción raise

Permite provocar una excepción

```
raise ValueError("Error de valor generado")  
# ValueError: Error de valor generado
```

Idea central

raise puede incluir o no un mensaje de error asociado. Sirve para señalar explícitamente que una condición no puede continuar.

raise para relanzar una excepción

También puede utilizarse para volver a lanzar la última excepción producida

```
try:
    num = int("abc")
except ValueError as msg:
    print("No se pudo guardar el número")
    print(msg)
    raise
```

Qué ocurre

Primero se hace un tratamiento local del error. Luego raise permite que la excepción siga su recorrido para ser vista o tratada más arriba.

Instrucción assert

Valida una condición lógica y genera AssertionError si es falsa

```
assert True
assert 5 > 3
assert False
# AssertionError

assert False, "Mensaje"
# AssertionError: Mensaje
```

Definición

Se utiliza junto con una operación lógica. Si la condición es falsa, genera una excepción del tipo AssertionError.

Jerarquía de excepciones

Los tipos de excepciones se organizan de forma jerárquica

```
try:
    capacidad_total = int(input("Stock máximo: "))
    stock_actual = int(input("Stock actual: "))
    if stock_actual > capacidad_total:
        raise OverflowError("El stock supera la capacidad máxima")
    porc_ocupado = stock_actual * 100 / capacidad_total
    print(f"El almacén está a un {porc_ocupado:.2f}% de su capacidad")
except ValueError:
    print("El valor ingresado es incorrecto")
except ArithmeticError as msg:
    print(msg)
```

Interpretación

Distintas excepciones pueden pertenecer a un tipo más genérico. Por eso el orden y el tipo de except importan.

Excepciones personalizadas

Se pueden generar nuevos tipos de excepción para usar con raise

```
class EmptyListError(Exception):  
    pass  
  
def promedio_lista(lista):  
    if len(lista) == 0:  
        raise EmptyListError("La lista está vacía")  
    else:  
        return sum(lista) / len(lista)
```

Si la lista no cumple la condición mínima para calcular el promedio, se lanza una excepción propia y explícita.